$$
\begin{array}{rl}
e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid () & \text{constants} \\
\mid \textbf{let}\ x = e_1\ \textbf{in}\ e_2 & \text{let binding} \\
\mid \textbf{uniform}(e_1, e_2) \mid \textbf{gaussian}(e_1, e_2) \mid \textbf{exponential}(e) \mid \ldots & \text{continuous distributions} \\
\mid \textbf{discrete}(p_0, \ldots, p_{n-1}) & \text{discrete distribution} \\
\mid e_1 < e_2 \mid e_1 \le e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \le_{\#n} e_2 & \text{comparisons} \\
\mid e_1 \,\&\&\, e_2 \mid e_1 \,||\, e_2 \mid \text{not } e & \text{Boolean operations} \\
\mid \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 & \text{conditional} \\
\mid (e_1, e_2) \mid \textbf{fst}\ e \mid \textbf{snd}\ e & \text{pairs} \\
\mid \textbf{fun}\ x \ \to\ e \mid e_1\ e_2 \mid \textbf{fix}\ f\ x := e & \text{functions} \\
\mid \textbf{observe}\ e \mid e_1; e_2 & \text{effects} \\
\mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match}\ e\ \textbf{with} \mid \text{nil} \to e_{\text{nil}} \mid e_1 :: e_2 \to e_{\text{cons}}\ \textbf{end}) & \text{lists} \\
\mid \textbf{diverge} & \text{diverge}
\end{array}
$$

Fig. 5. Syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct **observe**(e) conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual discrete distributions and any continuous distribution with a tractable cumulative distribution function (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). Adding further distributions is straightforward as long as their CDFs are known; §C lists the ones implemented.

The expression **discrete**$(p_0, \ldots, p_{n-1})$ requires a nonempty probability vector: $n \ge 1$, every $p_i \ge 0$, and $\sum_i p_i = 1$. It returns $k_{\#n}$ with probability p_k.

These are ideal real-valued premises. The implementation separately validates finite binary64 weights. It normalizes compiler-generated weights only within a tight allowance for accumulated CDF arithmetic, and parses source vectors within a documented 10^{-4} rounded-input allowance. These engineering policies do not replace the exact premise above: binary64 evaluation and normalization do not by themselves establish exact preservation of ideal real-valued probabilities, which requires a separate numerical error analysis.

3.2 Types

We formalize the types that Slice supports. A *value* set V is a finite set of reals or $\top$. Elements of $\{<x \mid x \in \mathbb{R}\} \cup \{\le x \mid x \in \mathbb{R}\}$ are called *cuts*. We order cuts first by their real boundary and then by their tag, with $<c \prec \le c$ at a shared boundary c. Thus every finite non-empty cut set has a unique enumeration $B = \{\sim_1 x_1, \ldots, \sim_n x_n\}$ in increasing tagged-cut order. For a cut $b = \sim x$, write $S_b = \{r \in \mathbb{R} \mid r \sim x\}$. The canonical order makes $S_{b_1} \subsetneq \cdots \subsetneq S_{b_n}$. We define the indexed partition induced by B by

$$\text{Intervals}(B)_0 = S_{b_1}, \qquad \text{Intervals}(B)_i = S_{b_{i+1}} \setminus S_{b_i}\ (0 < i < n), \qquad \text{Intervals}(B)_n = \mathbb{R} \setminus S_{b_n}.$$

In particular, $\text{Intervals}(\{<0.1, \le 0.5\}) = \{(-\infty, 0.1), [0.1, 0.5], (0.5, \infty)\}$. The strict and non-strict cuts differ only in which side contains the boundary point: $\text{Intervals}(\{<c\}) = \{(-\infty, c), [c, \infty)\}$, whereas $\text{Intervals}(\{\le c\}) = \{(-\infty, c], (c, \infty)\}$. Thus a boundary generated by $e < c$ places c on the

$$\textsc{Float-Const}\ \frac{\mathrm{has}(V, c)}{\Gamma \vdash c : \mathbf{float}[B; V]} \qquad \textsc{Continuous-1}\ \frac{\Gamma \vdash e : \mathbf{float}[B_1; V_1] \text{ and } \mathrm{recovers}(B_1, V_1)}{\Gamma \vdash \mathbf{exponential}(e) : \mathbf{float}[B; \top]}$$

$$\textsc{Continuous-2}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B_1; V_1] \text{ and } \mathrm{recovers}(B_1, V_1) \qquad \Gamma \vdash e_2 : \mathbf{float}[B_2; V_2] \text{ and } \mathrm{recovers}(B_2, V_2)}{\Gamma \vdash \mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; \top]}$$

$$\textsc{Less}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B; V_1] \qquad \Gamma \vdash e_2 : \mathbf{float}[B; V_2] \qquad \mathrm{answers}_{<}(B, V_1, V_2)}{\Gamma \vdash e_1 < e_2 : \mathbf{bool}}$$

$$\textsc{Less-Equal}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B; V_1] \qquad \Gamma \vdash e_2 : \mathbf{float}[B; V_2] \qquad \mathrm{answers}_{\leq}(B, V_1, V_2)}{\Gamma \vdash e_1 \leq e_2 : \mathbf{bool}}$$

Fig. 6. Typing rules for float types and continuous distributions (**uniform** is a representative for all other two-argument distributions). The notions *recovers* and *answers* are defined in Definitions 3.2, 3.3 and 3.4. Here Γ is a typing environment of the form $\{x_1 : \tau_1, \ldots, x_n : \tau_n\}$, where $\Gamma, x : \tau_1 = (\Gamma \setminus \{x : \tau \mid \tau \text{ type}\}) \cup \{x : \tau_1\}$.

right, while a boundary generated by $e \leq c$ places c on the left. Moreover, we let $\mathrm{Intervals}(\emptyset)_0 = \mathbb{R}$; $\mathrm{Intervals}(\top)$ is not used because $\top$ denotes an undiscretized representation. The construction above consists of $n + 1$ pairwise disjoint intervals whose union is $\mathbb{R}$, so every real has a unique interval index. We denote the set of all cut- and value sets, respectively, by $\mathcal{B}$ and $\mathcal{V}$.

Definition 3.1 (Types). For $n \in \mathbb{N}$, $B \in \mathcal{B}$, and $V \in \mathcal{V}$, types τ adhere to the grammar

$$\tau ::= \mathbf{int} \mid \mathbf{bool} \mid \mathbf{unit} \mid \mathbf{fin}(n) \mid \mathbf{float}[B; V] \mid \tau_1 * \tau_2 \mid \tau_1 \to \tau_2 \mid \mathbf{list}(\tau)\ .$$

Standard types cover integers, booleans, ordered pairs, functions, and lists. Finite types $\mathbf{fin}(n)$ contain $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$; the $\#n$-subscript reminds inference backends such as Dice that these values live in a finite domain. The float type $\mathbf{float}[B; V]$ annotates a real-valued expression with its cut set B (an over-approximation of the comparisons it participates in) and value set V (an over-approximation of the values it can take). $B = \top$ means the expression may be compared to arbitrary reals; $B = \emptyset$ means it will never be compared; $V = \top$ indicates an unrestricted result.

3.3 Typing Rules

The typing rules are split into three parts: Rules specific to float types and continuous distributions (Figure 6), standard rules for all other language constructs (Figure 7), and subtyping (Figure 8).

To understand the float typing rules, it is helpful to understand how discretization works. A type $\mathbf{float}[\mathrm{B}; \mathrm{V}]$ is discretized as follows: If $B \neq \top$ is a finite set of cuts, then the type is discretized to a type $\mathbf{fin}(n+1)$ where $n+1 = |\mathrm{Intervals}(B)|$ is the number of intervals induced by B. For instance, if $B = \{<0.5, \leq 0.8\}$, then the type is discretized to $\mathbf{fin}(3)$, where $0_{\#3}$ represents the interval $(-\infty, 0.5)$, $1_{\#3}$ represents the interval $[0.5, 0.8]$, and $2_{\#3}$ represents the interval $(0.8, \infty)$. If $B = \top$, then no discretization is performed, and the type remains $\mathbf{float}$.

To understand the typing rules, keep in mind that the cut sets B need to be sufficiently fine-grained — i.e., contain enough cuts — so that the discretized program has enough information to determine how the original continuous program behaved.

Constants. Rule Float-Const assigns a constant c the type $\mathbf{float}[B; V]$ whenever $\mathrm{has}(V, c)$ holds (i.e., $c \in V$ or $V = \top$). Constants do not constrain the cut set, but recording them in V ensures later comparisons know which values must remain distinguishable.

Continuous distributions. Rule Continuous-1 types a single-argument distribution when its argument e has type $\mathbf{float}[B; V]$ and $\mathrm{recovers}(B, V)$:

Definition 3.2. Let $B \in \mathcal{B}$ and $V \in \mathcal{V}$. We say that recovers(B, V) if $B = \top$ or ($V \neq \top$ and every interval I induced by B contains at most one value in V, i.e., $\forall I \in \text{Intervals}(B) : |V \cap I| \leq 1$).

Intuitively, recovers(B, V) means that the discretization induced by B is lossless on V: from the resulting interval index, the original value in V can be uniquely recovered. For example, **exponential**(e) can be discretized only when e takes finitely many recoverable values. Thus V must be finite and B must separate those values so the discretized program can determine which one occurred. §5 revisits this invariant. The rule for two-argument distributions is analogous.

Comparisons. For $e < c$ with $e : \textbf{float}[B_1; \top]$ and $c : \textbf{float}[B_2; \{c\}]$, the discretized e must still decide the predicate, so B_1 must include $< c$. If c can take any value in V_2, every $< v$ for $v \in V_2$ must appear. Symmetrically, for $c < e$ with $e : \textbf{float}[B_2; V_2]$, B_2 must contain $\leq c$ so the discretized intervals know on which side of c they fall. For example, $2.3 < e$ requires cuts separating $(-\infty, 2.3]$ from $(2.3, \infty)$; finer partitions are allowed but not required.

In general, the typing rule for $e_1 < e_2$ requires that the expressions share a cut set B, and that their respective value sets V_1, V_2 are related in a way that allows the comparison to be determined after discretization, via the answers$_<(B, V_1, V_2)$ relation:

Definition 3.3. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that answers$_<(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {<x} \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {\leq x} \in B$.

To understand this definition, we first want to introduce two different perspectives on the B set. The first perspective is that B specifies a set of intervals, such that values of the expression will be mapped to their respective interval index. The second perspective is that this discretized index *still contains enough information to evaluate all comparisons in B when viewed as predicates.* That is, if B contains $<x$ then the discretized value of $e : \textbf{float}[B; V]$ still contains enough information to answer $e < x$ precisely: if after discretization, e has an interval index below the interval containing x, then the comparison is true, and otherwise it is false.

Now, if $e_1 : \textbf{float}[B; V_1]$ and $e_2 : \textbf{float}[B; V_2]$, then answers$_<(B, V_1, V_2)$ ensures that the comparison result can still be determined after discretization because:

(1) If $B = \top$, then no discretization is performed, and the comparison can just be performed on the real-valued expression.
(2) If $B \neq \top$ and $V_2 \neq \top$, then the expression is discretized into intervals as determined by B, but in this case the right-hand side can only take on values in V_2. In this case, all $<x$ for $x \in V_2$ must be contained in B, which means that the value of e_2 can be exactly recovered from its interval index. Then, given the previous observation, we can determine the comparison result by comparing the interval index of e_1 to the interval index of e_2.
(3) If $B \neq \top$ and $V_1 \neq \top$, then the situation is similar to the previous case.

Note that $B = \top$ iff $V = \top$ accounts for comparisons where a continuously ranging expression is compared against arbitrary real values not statically known, such as other continuous distributions, in which case no discretization can be performed. Additionally, we insist that $B_1 = B_2$, which ensures that both comparands will be discretized to the same type, or not discretized at all.

Finally, for non-strict comparisons, the rule is analogous to Definition 3.3:

Definition 3.4. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that answers$_\leq(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {\leq x} \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {<x} \in B$.

The tagged-cut convention is essential here. If both $<c$ and $\leq c$ occur, their order isolates $\{c\}$ between them. More generally, membership in each predicate set S_b is determined by the interval index. This does not make a comparison constant on every arbitrary pair of induced intervals. Instead, the typing premises give the property needed on reachable values. For answers$_<(B, V_1, V_2)$,

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}$$

$$\textsc{Observe}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathbf{observe}\ e : \mathbf{unit}} \qquad \textsc{Discrete}\ \frac{n \geq 1 \qquad \forall i \in \{0, \ldots, n-1\}.\ p_i \geq 0 \qquad \sum_{i=0}^{n-1} p_i = 1}{\Gamma \vdash \mathbf{discrete}(p_0, \ldots, p_{n-1}) : \mathbf{fin}(n)}$$

$$\textsc{FinConst}\ \frac{0 \leq k < n}{\Gamma \vdash k_{\#n} : \mathbf{fin}(n)} \qquad \textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 \leq_{\#n} e_2 : \mathbf{bool}}$$

$$\textsc{Diverge}\ \frac{}{\Gamma \vdash \mathbf{diverge} : \tau}$$

Fig. 7. Typing rules for general language constructs

$$\textsc{Sub-Refl}\ \frac{\tau \in \{\mathbf{int}, \mathbf{bool}, \mathbf{fin}(n)\}}{\tau <: \tau} \qquad \textsc{Sub-Float}\ \frac{B_1 = B_2 \qquad V_1 \sqsubseteq V_2}{\mathbf{float}[B_1; V_1] <: \mathbf{float}[B_2; V_2]} \qquad \textsc{Sub-Pair}\ \frac{\tau_1 <: \tau_1' \qquad \tau_2 <: \tau_2'}{\tau_1 * \tau_2 <: \tau_1' * \tau_2'}$$

$$\textsc{Sub-List}\ \frac{\tau <: \tau'}{\mathbf{list}(\tau) <: \mathbf{list}(\tau')} \qquad \textsc{Sub-Arrow}\ \frac{\tau_1' <: \tau_1 \qquad \tau_2 <: \tau_2'}{\tau_1 \to \tau_2 <: \tau_1' \to \tau_2'} \qquad \textsc{Sub-Type}\ \frac{\tau_1 <: \tau_2 \qquad \Gamma \vdash e : \tau_1}{\Gamma \vdash e : \tau_2}$$

Fig. 8. Subtyping relation. For both cut- and value sets, we let $d \sqsubseteq d'$ iff $\begin{cases} d \subseteq d' & \text{if } d \neq \top \text{ and } d' \neq \top \\ d' = \top & \text{otherwise .} \end{cases}$

either the right operand is known to lie in V_2 and $<y \in B$ for every reachable $y \in V_2$, or the left operand is known to lie in V_1 and $\leq x \in B$ for every reachable $x \in V_1$. In the first case the interval index decides $x < y$ for every reachable pair, and in the second case it decides the same predicate symmetrically. Replacing $<y$ and $\leq x$ by the cuts in $\text{answers}_<(B, V_1, V_2)$ gives the corresponding reachable-pair statement for $x \leq y$.

Subtyping. The type system includes a subtyping relation that captures when one type can be replaced by another. The subtyping rules are shown in Figure 8. For float types $\mathbf{float}[B; V]$, subtyping follows the lattice ordering on V: an expression known to range over some set of values can also be considered to range over a superset of those values. For B, we require equality to make sure that the discretized representation of the expression is not changed by subtyping.

All other typing rules are standard and depicted in Figure 7 and §D, respectively.

4 Type Inference

We now describe our type inference algorithm, whose goal is to annotate each subexpression of a program with its most precise type. Specifically, we focus on expressions of type $\mathbf{float}[B; V]$ and show how Slice populates the cut set B and value set V, which later determine how and whether the expression can be discretized. Type inference for all other types is standard and follows the classic Hindley-Milner algorithm. For float-typed expressions, we use a two-phase approach: the first phase traverses the program once to generate constraints over cut and value sets; the second phase solves those constraints to produce a fully annotated program.

The distinction between typing and type inference is important. The correctness argument cleanly separates into two pieces: (1) type inference produces a typing derivation matching our

$e : \tau$	Condition	$\mathcal{D}[\![e : \tau]\!]$
$x : \tau$		x
$c : \texttt{float}[B; V]$	$B = \top$	c
	$B \neq \top$	$k_{\#(\lvert B\rvert+1)}$ such that $c \in \text{Intervals}(B)_k$
$\texttt{let}\ x = e_1 : \tau_1\ \texttt{in}\ e_2 : \tau_2 : \tau_2$		$\texttt{let}\ x = \mathcal{D}[\![e_1 : \tau_1]\!]\ \texttt{in}\ \mathcal{D}[\![e_2 : \tau_2]\!]$
$(\texttt{if}\ e_1 : \texttt{bool}\ \texttt{then}\ e_2 : \tau\ \texttt{else}\ e_3 : \tau) : \tau$		$\texttt{if}\ \mathcal{D}[\![e_1 : \texttt{bool}]\!]\ \texttt{then}\ \mathcal{D}[\![e_2 : \tau]\!]\ \texttt{else}\ \mathcal{D}[\![e_3 : \tau]\!]$
$(e_1 : \texttt{float}[B; V_1] < e_2 : \texttt{float}[B; V_2]) : \texttt{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] <_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] < \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
$(e_1 : \texttt{float}[B; V_1] \leq e_2 : \texttt{float}[B; V_2]) : \texttt{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] \leq_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] \leq \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
$\texttt{uniform}(e_1 : \texttt{float}[B_1; V_1],\ e_2 : \texttt{float}[B_2; V_2]) : \texttt{float}[B; V]$	$e_1, e_2 \in \mathbb{R},\ e_1 \geq e_2$	$\texttt{diverge} : \tau$
	$B = \top,\ \neg(e_1, e_2 \in \mathbb{R} \wedge e_1 \geq e_2)$	$\texttt{uniform}(\mathcal{D}[\![e_1 : \texttt{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \texttt{float}[B_2; V_2]]\!])$
	$e_1, e_2 \in \mathbb{R},\ e_1 < e_2,$ $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$	$\texttt{discrete}(p_0, \ldots, p_n)$ where $p_k = \Pr[X \in \text{Intervals}(B)_k],\ X \sim \text{Uniform}(e_1, e_2)$
	$B \neq \top,\quad (e_1 \notin \mathbb{R} \text{ or } e_2 \notin \mathbb{R}), \&\&$ $B_1 \neq \top,$ $B_2 \neq \top,$ $V_1 = \{v_1, \ldots, v_n\},$ $V_2 = \{w_1, \ldots, w_m\}$	$\texttt{let}\ v_1' = \mathcal{D}[\![e_1 : \texttt{float}[B_1; V_1]]\!]\ \texttt{in}$ $\texttt{let}\ v_2' = \mathcal{D}[\![e_2 : \texttt{float}[B_2; V_2]]\!]\ \texttt{in}$ $\texttt{if}\ v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_1 : \texttt{float}[B_1; V_1]]\!]$ $\&\&\ v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_1 : \texttt{float}[B_2; V_2]]\!]\ \texttt{then}$ $\quad \mathcal{D}[\![\texttt{uniform}(v_1, w_1) : \texttt{float}[B; V]]\!]$ $\ldots$ $\texttt{else if}\ v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_i : \texttt{float}[B_1; V_1]]\!]$ $\quad \&\&\ v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_j : \texttt{float}[B_2; V_2]]\!]\ \texttt{then}$ $\quad\quad \mathcal{D}[\![\texttt{uniform}(v_i, w_j) : \texttt{float}[B; V]]\!]$ $\ldots$ $\texttt{else}\ \mathcal{D}[\![\texttt{uniform}(v_n, w_m) : \texttt{float}[B; V]]\!]$

Fig. 13. Discretization function. The displayed cases are pairwise disjoint. Uniform distributions require a strict parameter ordering $e_1 < e_2$. The constant-parameter rule assigns each canonical tagged-cut interval its probability mass. For a general law and tagged cut $b = {\sim}c$, the corresponding cumulative boundary is $H(b) = \Pr[X \sim c]$, so $H(<c) = \Pr[X < c]$ and $H(\leq c) = \Pr[X \leq c]$. For the strict uniform family these agree at each finite boundary because the law is atomless. For the **uniform** distribution, $V = \top$ is implicit, and all remaining cut-set and value-set cases are ruled out by the constraints enforced during type inference. For all other language constructs, $\mathcal{D}[\![\cdot]\!]$ recurses structurally over them.

can be *discretized* to an expression $e' : \texttt{fin}(n+1)$ such that the probability that e evaluates to a value in $\text{Intervals}(B)_i$ *coincides* with the probability that e' evaluates to $i_{\#n+1}$. In particular, if *all* $\texttt{float}[B; V]$-typed subexpressions are finite-cut, then the given expression can be *fully discretized*.

Example 5.1. Consider a uniform distribution whose outcomes are compared to constants:

```
let x = uniform(0, 1) : float[B; T] in
  (x : float[B; T] < (0.3 : float[B; {0.3}])) ||
   (x : float[B; T] >= (0.5 : float[B; {0.5}]))
```

where $B = \{<0.3, <0.5\}$. The cut set B informs us that the probability of the comparisons evaluating to true is determined by the probability of x evaluating to one of the intervals

$$\text{Intervals}(B)_0 = (-\infty, 0.3) \qquad \text{Intervals}(B)_1 = [0.3, 0.5) \qquad \text{Intervals}(B)_2 = [0.5, \infty)$$

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}$$

$$\textsc{Discrete}\ \frac{n \geq 1 \qquad \forall i \in \{0, \ldots, n-1\}.\ p_i \geq 0 \qquad \sum_{i=0}^{n-1} p_i = 1}{\Gamma \vdash \mathbf{discrete}(p_0, \ldots, p_{n-1}) : \mathbf{fin}(n)} \qquad \textsc{LessEq}\ \frac{\Gamma \vdash e : \mathbf{int}}{\Gamma \vdash e \leq i : \mathbf{bool}}$$

$$\textsc{Pair}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 * \tau_2} \qquad \textsc{Fst}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \mathbf{fst}\ e : \tau_1} \qquad \textsc{Snd}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \mathbf{snd}\ e : \tau_2} \qquad \textsc{Fun}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \mathbf{fun}\ x\ \to\ e : \tau_1 \to \tau_2}$$

$$\textsc{App}\ \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2} \qquad \textsc{True}\ \frac{}{\Gamma \vdash \mathrm{true} : \mathbf{bool}} \qquad \textsc{False}\ \frac{}{\Gamma \vdash \mathrm{false} : \mathbf{bool}}$$

$$\textsc{And}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \mathbf{bool}}{\Gamma \vdash e_1\ \&\&\ e_2 : \mathbf{bool}} \qquad \textsc{Or}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \mathbf{bool}}{\Gamma \vdash e_1\ ||\ e_2 : \mathbf{bool}} \qquad \textsc{Not}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathrm{not}\ e : \mathbf{bool}}$$

$$\textsc{Unit}\ \frac{}{\Gamma \vdash () : \mathbf{unit}} \qquad \textsc{FinConst}\ \frac{0 \leq k < n}{\Gamma \vdash k_{\#n} : \mathbf{fin}(n)} \qquad \textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \mathbf{bool}}$$

$$\textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 \leq_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{Observe}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathbf{observe}\ e : \mathbf{unit}} \qquad \textsc{Seq}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1; e_2 : \tau_2}$$

$$\textsc{Fix}\ \frac{\Gamma, f : \tau_1 \to \tau_2, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \mathrm{fix}\ f\ x := e : \tau_1 \to \tau_2} \qquad \textsc{Nil}\ \frac{}{\Gamma \vdash \mathrm{nil} : \mathbf{list}(\tau)} \qquad \textsc{Cons}\ \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \mathbf{list}(\tau)}{\Gamma \vdash e_1 :: e_2 : \mathbf{list}(\tau)}$$

$$\textsc{Match}\ \frac{\Gamma \vdash e : \mathbf{list}(\tau_1) \qquad \Gamma \vdash e_1 : \tau_2 \qquad \Gamma, h : \tau_1, t : \mathbf{list}(\tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \mathrm{match}\ e\ \mathrm{with\ nil} \to e_1 \mid h :: t \to e_2\ \mathrm{end} : \tau_2}$$

$$\textsc{Diverge}\ \frac{}{\Gamma \vdash \mathbf{diverge} : \tau}$$

Fig. 25. Typing rules for general language constructs

Case $e = (e_1 : \mathbf{float}[B; V_1] \leq e_2 : \mathbf{float}[B; V_2])$. Analogous to the $<$ case, using $\mathrm{answers}_{\leq}(B, V_1, V_2)$ and FinLessEq.

Case $e = (\mathbf{uniform}(e_1 : \mathbf{float}[B_1; V_1],\ e_2 : \mathbf{float}[B_2; V_2]) : \mathbf{float}[B; \top])$. The Continuous-2 rule was applied, so recovers(B_1, V_1) and recovers(B_2, V_2) hold. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \mathbf{float}[B_i; V_i]]\!] : \mathcal{D}[\![\mathbf{float}[B_i; V_i]]\!]$ for $i = 1, 2$. We consider each sub-case of the discretization function:

- $e_1, e_2 \in \mathbb{R}$ and $e_1 \geq e_2$: then $\mathcal{D}[\![\mathbf{uniform}(\ldots) : \mathbf{float}[B; \top]]\!] = \mathbf{diverge}$, which is well-typed at any type by Diverge.
- $B = \top$ and the literal-invalid condition above does not hold: then $\mathcal{D}[\![\mathbf{float}[B; \top]]\!] = \mathbf{float}$, and

$$\mathcal{D}[\![\mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; \top]]\!] = \mathbf{uniform}(\mathcal{D}[\![e_1 : \mathbf{float}[B_1; V_1]]\!],\ \mathcal{D}[\![e_2 : \mathbf{float}[B_2; V_2]]\!]).$$

  Since recovers(B_1, V_1) and recovers(B_2, V_2) are preserved under discretization, the result is well-typed at $\mathbf{float}$ by Continuous-2.
- $B \neq \top$, $e_1, e_2 \in \mathbb{R}$, and $e_1 < e_2$: then $\mathcal{D}[\![\mathbf{float}[B; \top]]\!] = \mathbf{fin}(|B| + 1)$, and

$$\mathcal{D}[\![\mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; \top]]\!] = \mathbf{discrete}(p_0, \ldots, p_{|B|})$$

  where $p_k = H(b_{k+1}) - H(b_k)$ for the tagged cumulative function $H(\sim c) = \Pr[X \sim c]$, with artificial outer values 0 and 1. This sub-case has $e_1 < e_2$ because the preceding rule handles equality and reversed bounds. By the canonical tagged-cut order, H is nondecreasing; hence each $p_k = \Pr[X \in \text{Intervals}(B)_k] \in [0, 1]$ and the differences telescope to 1. Thus the result is well-typed at $\mathbf{fin}(|B| + 1)$ by Discrete.
- $B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, and at least one of e_1, e_2 is nonliteral: the algorithm generates conditionals over all pairs (v_i, w_j), each branch dispatching to a constant parameter in $\mathbf{uniform}$. By the previous sub-case, each valid branch $\mathcal{D}[\![\mathbf{uniform}(v_i, w_j) : \mathbf{float}[B; \top]]\!]$ is well-typed at $\mathbf{fin}(|B| + 1) = \mathcal{D}[\![\mathbf{float}[B; \top]]\!]$, while equality or reversed bounds use the malformed branch. The conditional structure is well-typed by If (as in the $\mathbf{if}$ case above), so the full expression is well-typed at $\mathcal{D}[\![\mathbf{float}[B; \top]]\!]$.

Case e is any other language construct (functions, pairs, lists, *etc.*). By definition of $\mathcal{D}[\![\cdot]\!]$, the discretization recurses structurally over e and τ. That is, $\mathcal{D}[\![e : \tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each subexpression and subtype of $e : \tau$, and $\mathcal{D}[\![\tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each component type of τ. The result then follows immediately by applying the inductive hypothesis to each subterm, analogous to the above cases.

In every case, the typing derivation is preserved under $\mathcal{D}[\![\cdot]\!]$. Therefore,

$$\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]. \qquad \square$$

F Soundness Proof

F.1 Measure Theory Preliminaries

We first give a brief overview of basic measure-theoretic terms and lemmas, and refer the reader to standard textbooks for further background [43, 44].

Topological space. A topological space is a pair $(X, \mathcal{T})$ where $\mathcal{T}$ is a collection of subsets of X, called open sets, such that $\emptyset, X \in \mathcal{T}$, and $\mathcal{T}$ is closed under arbitrary unions and finite intersections.

σ-algebra. A σ-algebra on a set X is a collection Σ of subsets of X that includes X itself and is closed under complement and under countable unions. By implication, a σ-algebra includes the empty set $\emptyset$ and is closed under countable intersections. Given a topological space $(X, \mathcal{T})$, the Borel σ-algebra $\mathcal{B}(X)$ is the smallest σ-algebra containing all open sets in $\mathcal{T}$.

Equipped with these notions, we define the small-step semantics of our language:

Definition F.34. The small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr_\tau + \bot)$ is defined recursively on the structure of $e : \tau$ by the rules in Table 1 and Table 2.

In the Discrete row, the typing premises ensure that the displayed weights form a nonempty probability vector. Hence $\mathrm{Discrete}(p_0, \ldots, p_{n-1}) = \sum_{i=0}^{n-1} p_i \, \delta_{i_{\mathbb{R}^n}}$ is a probability measure in $\mathcal{D}(Expr_{\mathtt{fin}(n)} + \bot)$. The missing probability mass in $[\![e : \tau]\!]$ is the probability of encountering an error due to ill-formed parameters fed into distributions (e.g., $\mathtt{uniform}(0.8, 0.1)$). $[\![e : \tau]\!](\bot)$ is the probability of encountering an observation failure.

Lemma F.35 (Measurability of constant extension map). *Let $A \subseteq \mathbb{R}^n$ be equipped with the subspace σ-algebra*

$$\mathcal{F}_A := \{A \cap M \mid M \in \mathcal{B}(\mathbb{R}^n)\},$$

and let $B \subseteq \mathbb{R}^d$ be equipped with the subspace σ-algebra

$$\mathcal{F}_B := \{B \cap N \mid N \in \mathcal{B}(\mathbb{R}^d)\}.$$

Fix constant vectors $c_1 \in \mathbb{R}^{m_1}, \ldots, c_k \in \mathbb{R}^{m_k}$. Define

$$g : \mathbb{R}^n \to \mathbb{R}^d$$

to be any map obtained by concatenating blocks consisting of

- *copies of the input vector $\vec{r} \in \mathbb{R}^n$, and*
- *the constant vectors $c_1, \ldots, c_k$,*

in some fixed order. That is, $g(\vec{r})$ is formed by inserting the vectors $\vec{r}$ and c_j into a fixed sequence of blocks.

Assume $g(A) \subseteq B$ and define $\phi := g|_A : A \to B$. Then ϕ is measurable.

Example F.36 (Constant extension maps). The constant extension map allows constants to be inserted and the input vector $\vec{r} \in \mathbb{R}^n$ to appear multiple times. The following are examples of such maps:

- $g : \mathbb{R}^n \to \mathbb{R}^n, \quad g(\vec{r}) = \vec{r}$
- $g : \mathbb{R}^n \to \mathbb{R}^{n+m_1}, \quad g(\vec{r}) = (\vec{r}, c_1), \quad c_1 \in \mathbb{R}^{m_1}$
- $g : \mathbb{R}^n \to \mathbb{R}^{m_1+n+m_2}, \quad g(\vec{r}) = (c_1, \vec{r}, c_2), \quad c_1 \in \mathbb{R}^{m_1}, \ c_2 \in \mathbb{R}^{m_2}$
- $g : \mathbb{R}^n \to \mathbb{R}^{n+m_2}, \quad g(\vec{r}) = (\vec{r}, c_2), \quad c_2 \in \mathbb{R}^{m_2}$
- $g : \mathbb{R}^n \to \mathbb{R}^{m_1+n+m_2+n+m_3+n}, \quad g(\vec{r}) = (c_1, \vec{r}, c_2, \vec{r}, c_3, \vec{r}), \quad c_i \in \mathbb{R}^{m_i}$

Proof. We show that $\phi^{-1}(S) \in \mathcal{F}_A$ for each $S \in \mathcal{F}_B$.

Fix $S \in \mathcal{F}_B$. By definition of $\mathcal{F}_B$, there exists $N \in \mathcal{B}(\mathbb{R}^d)$ such that $S = B \cap N$. We compute:

$$\begin{aligned}
\phi^{-1}(S) &= \{\vec{r} \in A \mid \phi(\vec{r}) \in B \cap N\} \\
&= \{\vec{r} \in A \mid \phi(\vec{r}) \in B \cap \phi(\vec{r}) \in N\} && \text{(definition of } \cap) \\
&= \{\vec{r} \in A \mid \phi(\vec{r}) \in N\} && (\phi(\vec{r}) \in B \text{ for all } \vec{r} \in A) \\
&= \{\vec{r} \in A \mid g(\vec{r}) \in N\} && \text{(definition of } \phi) \\
&= \{\vec{r} \in A \mid g(\vec{r}) \in N\} \cap \mathbb{R}^n && (\vec{r} \text{ ranges over } A \subseteq \mathbb{R}^n) \\
&= A \cap \{\vec{r} \in \mathbb{R}^n \mid g(\vec{r}) \in N\}. && \text{(rewrite as } \cap \text{ with } A)
\end{aligned}$$

It remains to show that $\{\vec{r} \in \mathbb{R}^n \mid (\vec{r}, \vec{c}) \in N\} \in \mathcal{B}(\mathbb{R}^n)$. Define the total constant-extension map

$$g\colon \mathbb{R}^n \to \mathbb{R}^d.$$

We claim g is continuous. A map into a product space is continuous if and only if each coordinate function is continuous [36, Theorem 19.6]. The coordinate functions of g are:

$\mathbf{e} : \boldsymbol{\tau}$	constraints	$[\![\boldsymbol{e} : \boldsymbol{\tau}, \mathcal{D}[\![\boldsymbol{e}]\!] : \mathcal{D}[\![\boldsymbol{\tau}]\!]]\!]$
$\mathtt{diverge} : \tau$		$\lambda A.0$
$v : \tau$		$\delta_{(v:\tau, \mathcal{D}[\![v:\tau]\!]:\mathcal{D}[\![\tau]\!])}$
$\mathtt{let}\ x = e_1 : \tau_1\ \mathtt{in}\ e_2 : \tau_2 : \tau_2$	$e_1 = v$ e_1 no val.	$\delta_{(e_2[v/x]:\tau_2, \mathcal{D}[\![e_2:\tau_2]\!][\mathcal{D}[\![v:\tau_1]\!]/x]:\mathcal{D}[\![\tau_2]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathtt{let}\ x=g\ \mathtt{in}\ e_2:\tau_2:\tau_2, \mathtt{let}\ x=g'\ \mathtt{in}\ \mathcal{D}[\![e_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]:\mathcal{D}[\![\tau_2]\!])}$
$\mathtt{if}\ e_1 : \mathtt{bool}\ \mathtt{then}\ e_2 : \tau\ \mathtt{else}\ e_3 : \tau : \tau$	$e_1 = \mathtt{true}$ $e_1 = \mathtt{false}$ o.w.	$\delta_{(e_2:\tau, \mathcal{D}[\![e_2:\tau]\!]:\mathcal{D}[\![\tau]\!])}$ $\delta_{(e_3:\tau, \mathcal{D}[\![e_3:\tau]\!]:\mathcal{D}[\![\tau]\!])}$ $[\![e_1 : \mathtt{bool}, \mathcal{D}[\![e_1 : \mathtt{bool}]\!] : \mathcal{D}[\![\mathtt{bool}]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathtt{if}\ g\ \mathtt{then}\ e_2:\tau\ \mathtt{else}\ e_3:\tau:\tau, \mathtt{if}\ g'\ \mathtt{then}\ \mathcal{D}[\![e_2:\tau]\!]:\mathcal{D}[\![\tau]\!]\ \mathtt{else}\ \mathcal{D}[\![e_3:\tau]\!]:\mathcal{D}[\![\tau]\!]:\mathcal{D}[\![\tau]\!])}$
$\mathtt{observe}\ (e : \mathtt{bool}) : \mathtt{unit}$	$e = \mathtt{true}$ $e = \mathtt{false}$ o.w.	$\delta_{(\mathtt{unit},\mathtt{unit})}$ $\delta_{\bot}$ $[\![e : \mathtt{bool}, \mathcal{D}[\![e]\!] : \mathtt{bool}]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathtt{observe}\ (g:\mathtt{bool}):\mathtt{unit}, \mathtt{observe}\ (g':\mathtt{bool}):\mathtt{unit})}$
$e_1 : \tau_1 < e_2 : \tau_2 : \mathtt{bool}$ for $\tau_1 = \mathtt{float}[B; V_1], \tau_2 = \mathtt{float}[B; V_2]$ $B = \top$	$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{(\mathtt{true}:\mathtt{bool}, \mathtt{true}:\mathtt{bool})}$ $\delta_{(\mathtt{false}:\mathtt{bool}, \mathtt{false}:\mathtt{bool})}$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2 : \tau_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(v_1:\tau_1 < g:\mathtt{bool}, \mathcal{D}[\![v_1:\tau_1]\!]:\mathcal{D}[\![\tau_1]\!] < g':\mathcal{D}[\![\mathtt{bool}]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(g < e_2:\tau_2:\mathtt{bool}, g' < \mathcal{D}[\![e_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]:\mathcal{D}[\![\mathtt{bool}]\!])}$
$e_1 : \tau_1 < e_2 : \tau_2 : \mathtt{bool}$ for $\tau_1 = \mathtt{float}[B; V_1], \tau_2 = \mathtt{float}[B; V_2]$ $B \neq \top$	$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{(\mathtt{true}:\mathtt{bool}, \mathtt{true}:\mathtt{bool})}$ $\delta_{(\mathtt{false}:\mathtt{bool}, \mathtt{false}:\mathtt{bool})}$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2 : \tau_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(v_1:\tau_1 < g:\mathtt{bool}, \mathcal{D}[\![v_1:\tau_1]\!]:\mathcal{D}[\![\tau_1]\!] <_{\#(\lvert B\rvert+1)} g':\mathcal{D}[\![\mathtt{bool}]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(g < e_2:\tau_2:\mathtt{bool}, g' <_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!]:\mathcal{D}[\![\mathtt{bool}]\!])}$
$\mathtt{uniform}(e_1 : \tau_1, e_2 : \tau_2) : \mathtt{float}[B; V]$ $B = \top$	$e_1 = v_1 < v_2 = e_2$ $e_1 = v_1 \geq v_2 = e_2$ $e_1 = v_1, e_2$ no val. e_1 no val.	$\mathrm{Uniform}(v_1, v_2) \gg\!= \lambda v.\delta_{(v:\mathtt{float}[B;V], v:\mathtt{float}[B;V])}$ $[\![\mathtt{diverge} : \tau, \mathcal{D}[\![\mathtt{uniform}(v_1 : \tau_1, v_2 : \tau_2)]\!] : \mathcal{D}[\![\tau]\!]]\!]$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathtt{uniform}(v_1:\tau_1, g:\tau_2):\tau, \mathtt{uniform}(\mathcal{D}[\![v_1]\!]:\mathcal{D}[\![\tau_1]\!], g':\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathtt{uniform}(g:\tau_1, e_2:\tau_2):\tau, \mathtt{uniform}(g':\mathcal{D}[\![\tau_1]\!], \mathcal{D}[\![e_2]\!]:\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau]\!])}$
$\mathtt{uniform}(e_1 : \tau_1, e_2 : \tau_2) : \mathtt{float}[B; V]$ $B \neq \top$ $\tau_1 = \mathtt{float}[B_1 \neq \top, \{v_1, \ldots, v_n\}]$ $\tau_2 = \mathtt{float}[B_2 \neq \top, \{w_1, \ldots, w_m\}]$	$e_1 = c_1 < c_2 = e_2$ values $e_1 = v_1 \geq v_2 = e_2$ $e_1 = c_1, e_2$ no val. e_1 no val.	$\mathrm{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{(v:\mathtt{float}[B;V], v_i:\mathtt{fin}(\lvert B\rvert+1))}$ where v_i is unique number s.t. $v \in \mathrm{Intervals}(B)_{v_i}$ $[\![\mathtt{diverge} : \tau, \mathcal{D}[\![\mathtt{uniform}(v_1 : \tau_1, v_2 : \tau_2)]\!] : \mathcal{D}[\![\tau]\!]]\!]$ $[\![e_2 : \tau_2, \mathcal{D}[\![e_2]\!] : \mathcal{D}[\![\tau_2]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathtt{uniform}(v_1:\tau_1, g:\tau_2):\tau, \mathtt{DC}_{B,B_1,B_2,V_1,V_2}(\mathcal{D}[\![v_1]\!]:\mathcal{D}[\![\tau_1]\!], g':\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau]\!])}$ $[\![e_1 : \tau_1, \mathcal{D}[\![e_1]\!] : \mathcal{D}[\![\tau_1]\!]]\!]$ $\gg\!= \lambda(g, g').\delta_{(\mathtt{uniform}(g:\tau_1, e_2:\tau_2):\tau, \mathtt{DC}_{B,B_1,B_2,V_1,V_2}(g':\mathcal{D}[\![\tau_1]\!], \mathcal{D}[\![e_2]\!]:\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau]\!])}$

Table 3. Small-step coupling semantics. Notice that $\mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]$ is uniquely determined by $e : \boldsymbol{\tau}$ so that it suffices to indicate the latter in the left-hand column. Here, every function of the form $f(g, g') = \lambda(g, g').\delta_{\ldots}$ is extended to $f_\bot$ in the sense of Definition F.29.

First observe that we have

$$\begin{aligned}
&\mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!] \\
&= \mathtt{DC}_{B,B_1,B_2,V_1,V_2}(\mathcal{D}[\![c_1 : \mathtt{float}[B_1; V_1]]\!], \mathcal{D}[\![c_2 : \mathtt{float}[B_2; V_2]]\!]) : \mathtt{fin}(|B| + 1) \\
&= \mathtt{DC}_{B,B_1,B_2,V_1,V_2}(k_1 : \mathtt{fin}(|B_1| + 1), k_2 : \mathtt{fin}(|B_2| + 1)) : \mathtt{fin}(|B| + 1)\,,
\end{aligned}$$

where k_i is the unique number such that $c_i \in \mathrm{Intervals}(B_i)_{k_i}$. By Table 1 and Figure 13, this yields

$$\begin{aligned}
&[\![\mathcal{D}[\![e : \tau]\!]]\!] \\
&= [\![\mathtt{discrete}(p_0, \ldots, p_n)]\!] \text{ where } p_k = \Pr[X \in \mathrm{Intervals}(B)_k] = H(b_{k+1}) - H(b_k), \quad X \sim \mathrm{Uniform}(c_1, c_2)\,.
\end{aligned}$$

Thus each p_k is the difference of F_{c_1,c_2} at consecutive cut locations (with endpoints $-\infty$ and $+\infty$). Strict and non-strict cuts assign a boundary point to different adjacent intervals, but this does not change these masses because $c_1 < c_2$ makes the uniform distribution atomless.

For Lemma F.43.2, consider the following:

$$
\begin{aligned}
& \pi_1(\llbracket e : \tau, \mathcal{D}\llbracket e \rrbracket : \mathcal{D}\llbracket \tau \rrbracket \rrbracket) \\
= \; & \llbracket e : \tau, \mathcal{D}\llbracket e \rrbracket : \mathcal{D}\llbracket \tau \rrbracket \rrbracket \gg\!= \lambda(e_1, e_2).\delta_{e_1} && \text{(definition)} \\
= \; & (\text{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{(v:\mathbf{float}[B;V], v_i:\mathbf{fin}(|B|+1))}) \gg\!= \lambda(e_1, e_2).\delta_{e_1} \\
& && \text{(Table 3, } v_i \text{ is unique number s.t. } v \in \text{Intervals}(B)_{v_i}) \\
= \; & \text{Uniform}(c_1, c_2) \gg\!= \lambda v.(\delta_{(v:\mathbf{float}[B;V], v_i:\mathbf{fin}(|B|+1))} \gg\!= \lambda(e_1, e_2).\delta_{e_1}) \\
& && \text{(monad assoc., } v_i \text{ is unique number s.t. } v \in \text{Intervals}(B)_{v_i}) \\
= \; & \text{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{v:\mathbf{float}[B;V]} \\
= \; & \text{Uniform}(c_1, c_2) \\
= \; & \llbracket e : \tau \rrbracket . && \text{(Table 1)}
\end{aligned}
$$

For Lemma F.43.2, consider the following:

$$
\begin{aligned}
& \pi_2(\llbracket e : \tau, \mathcal{D}\llbracket e \rrbracket : \mathcal{D}\llbracket \tau \rrbracket \rrbracket) \\
= \; & \llbracket e : \tau, \mathcal{D}\llbracket e \rrbracket : \mathcal{D}\llbracket \tau \rrbracket \rrbracket \gg\!= \lambda(e_1, e_2).\delta_{e_2} && \text{(definition)} \\
= \; & (\text{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{(v:\mathbf{float}[B;V], v_i:\mathbf{fin}(|B|+1))}) \gg\!= \lambda(e_1, e_2).\delta_{e_2} \\
& && \text{(Table 3, } v_i \text{ is unique number s.t. } v \in \text{Intervals}(B)_{v_i}) \\
= \; & \text{Uniform}(c_1, c_2) \gg\!= \lambda v.(\delta_{(v:\mathbf{float}[B;V], v_i:\mathbf{fin}(|B|+1))} \gg\!= \lambda(e_1, e_2).\delta_{e_2}) \\
& && \text{(monad assoc., } v_i \text{ is unique number s.t. } v \in \text{Intervals}(B)_{v_i}) \\
= \; & \text{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{v_i:\mathbf{fin}(|B|+1)} \\
& && \text{(monad return, } v_i \text{ is unique number s.t. } v \in \text{Intervals}(B)_{v_i}) \\
= \; & \llbracket \mathbf{discrete}(p_0, \ldots, p_n) \rrbracket \text{ where } p_k = \Pr[X \in \text{Intervals}(B)_k] = H(b_{k+1}) - H(b_k) \\
& && \text{(pushforward along the unique interval-index map)} \\
= \; & \llbracket \mathcal{D}\llbracket e \rrbracket : \mathcal{D}\llbracket \tau \rrbracket \rrbracket . && \text{(Table 1)}
\end{aligned}
$$

Now, the remaining cases are either (i) trivial base cases or (ii) immediately follow from the I.H. and Lemma F.44 further below. We demonstrate this for the case

$$
e : \tau = \mathbf{uniform}(e_1 : \mathbf{float}[B_1; V_1], e_2 : \mathbf{float}[B_2; V_2]) : \mathbf{float}[B; V] ,
$$

where

(1) $e_1 = c_1$ is a value but e_2 is no value,
(2) $B = \{\sim_1 b_1, \ldots, \sim_n b_n\}$,
(3) $V = \top$,
(4) $V_1 = \{v_1, \ldots, v_{n_1}\}$ with recovers(B_1, V_1) and $c_1 \in V_1$ due to has(V_1, c_1) by I.H.
(5) $V_2 = \{w_1, \ldots, w_{n_2}\}$ with recovers(B_2, V_2) and $c_2 \in V_2$ due to has(V_2, c_2) by I.H.